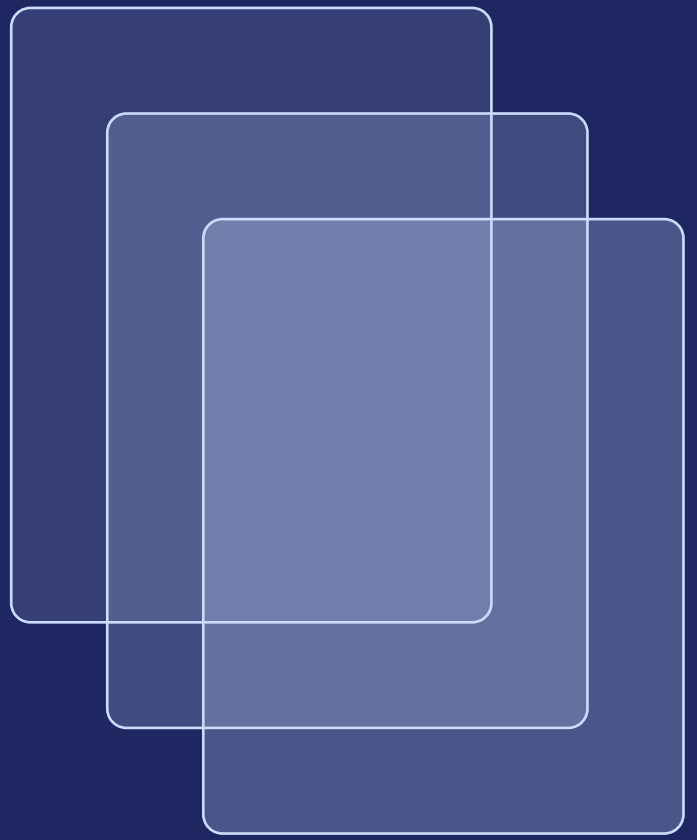
INTERNAL PROJECT PROPOSAL

# Dossier

Collect the documents, audit them, and prove the result — one configurable system

*From manual, one-off prompt runs to a measured, reproducible engine anyone on the team can run*



Management review · August 2026

THE STARTING POINT

# Where we are today

Document analysis works — but every audit is rebuilt by hand, and nothing tells us whether it was right.

### Collected by hand

Files are exported and downloaded manually, then dropped into a folder before anything can start.

### No way to tell if the output is right

Errors are found by reading results one at a time. We then adjust the prompt by instinct, with nothing to confirm it helped.

### Nothing is traceable or repeatable

A result cannot be tied back to a prompt version, a model and a source. Re-running months later can return a different answer.

### Every new use case starts over

A new type of audit — or a new source system — means rebuilding the whole flow.

THE SHIFT

# Sources and prompts become configuration. Quality becomes a number.

One tested system collects the documents and runs the audit. What changes between use cases is where the documents come from and what we ask of them. What never changes is that every result is scored against answers a human has verified.

### Any source, same handling

A shared folder, a Jira query or a wiki space all arrive in the same shape.

### A new audit in an afternoon

Three prompt files and a short configuration block.

### Every figure carries its evidence

Each extracted value points at the page and sentence it came from.

ARCHITECTURE

# Two halves, one boundary

Collecting documents touches the network. Auditing them must never do so — that is what makes a run repeatable.

HASH BOUNDARY

01

## Sync

Ask each source what changed and pull only that. Credentials and network live here, nowhere else.

02

## Store

Write each item to an immutable snapshot named by its content hash. Nothing is ever overwritten.

03

## Convert

Turn snapshots into markdown or page images. Cached on the snapshot hash plus the converter version.

04

## Audit

Extract, verify, score and report — working only from local, frozen files.

*Left of the line: mutable, remote, cannot be repeated exactly.  Right of the line: local, frozen, identical every time it runs.*

RETRIEVAL

# Three kinds of source, one interface

Each connector answers the same three questions: what exists, what changed, and give me this item.

FOLDER

## Manual drop folder

- For anything with no API, or a one-off export.
- A file is new when its content hash is new — re-dropping the same file changes nothing.
- Still snapshotted, so a manual audit replays like any other.

API

## Jira connector

- A JQL query defines the scope, e.g. all incidents in a project.
- Issue fields arrive as JSON; attachments are versioned separately from the issue.
- Only issues updated since the last sync are fetched.

API + PDF

## Confluence connector

- A space and a label define the scope.
- Pages are exported as PDF so diagrams and screenshots survive intact.
- Converted to markdown plus extracted page images, kept side by side.

CONFIGURATION

# What you actually write

Two files and a folder of prompts. This is the whole of a new use case.

**sources.yaml**

```yaml
sources:
  - id: incidents-manual
    type: folder
    path: inbox/incidents/
  - id: incidents-jira
    type: jira
    query: project = INC AND type = Incident
    fields: [key, summary, status, created,
             resolution, description, comments]
    attachments: true
    watermark_overlap: 24h

  - id: procedures-confluence
    type: confluence
    space: OPS
    label: procedure
    export: pdf        # keeps the screenshots
    convert: [markdown, page_images]
```

**pipelines/incident-audit/pipeline.yaml**

```yaml
sources: [incidents-jira, incidents-manual]
reference: [procedures-confluence]
gold: gold/incidents/

steps:
  - id: extract
    prompt: prompts/01_extract.md
    input: documents
    mode: per_file
    output: schemas/incident.json

  - id: verify
    prompt: prompts/02_verify.md
    input: [documents, extract, reference]
    mode: per_file

  - id: report
    prompt: prompts/03_report.md
    input: verify
    mode: aggregate
```

INCREMENTAL SYNC

# Only fetch what changed

A ledger records what we already hold. Tomorrow's 420 issues cost 20 downloads, not 420.

| SOURCE | ITEM | REMOTE VERSION | CONTENT HASH | STATE |
|---|---|---|---|---|
| incidents-jira | INC-4821 | `2026-07-30 09:14` | `a3f9c1e2` | unchanged |
| incidents-jira | INC-4822 | `2026-08-05 16:02` | `77c1b40a` | re-fetched |
| incidents-jira | INC-4901 | `2026-08-06 08:41` | `0e52dd18` | new |
| procedures-confluence | OPS-114 | `v37` | `b81f0c6d` | unchanged |
| incidents-manual | audit-note.pdf | — | `c4a70b93` | unchanged |

### Ask the server first

Jira and Confluence both accept an "updated since" filter — one query returns the twenty that moved.

### Fall back to hashing

Where a source exposes no version marker, we hash the payload and compare.

### Overlap the window

We re-ask from 24 hours before the last sync. Clock skew and indexing lag would otherwise drop items silently.

SAFEGUARDS

# What protects the retrieval

Four failure modes we have designed against, because each one is silent when it happens.

### Normalise before hashing

Source payloads carry fields that change on every fetch — rendered HTML, view counts, server timestamps. We hash only the fields we actually use, or everything looks modified every night.

### Version attachments separately

An issue can be untouched while the PDF attached to it was replaced. Each attachment carries its own hash, so a swapped file is detected even when the record around it is identical.

### Tombstone, never delete

An item that leaves the query scope is marked closed in the ledger, not removed. A run from March must still be able to resolve the documents it used.

### Runs pin their snapshots

A run records the exact content hashes it consumed. Replaying it never touches the network, and the system warns you when a source has moved since your last run.

THE PIPELINE

# The audit itself

Four stages. The same four every time, whatever the document or the question.

01

## Ingest

Snapshots become markdown, or page images where the visuals matter.

02

## Extract

One pass per document pulls out the requested fields as structured data, each one pointing at its evidence.

03

## Verify

A second pass re-reads the original alongside the extraction, checks the points we care about and corrects them.

04

## Report

A final pass reads the whole verified set and produces the summary that goes to the reader.

*Stage 03 is what makes the result trustworthy: the checker always sees the original document, never only the previous answer.*

GROUNDING

# Every figure carries its evidence

Nothing is extracted without a pointer to the page and sentence it came from.

extract / INC-4821.json

```
{
  "field": "loss_amount",
  "value": 48200.00,
  "currency": "CHF",
  "source": {
    "document": "INC-4821",
    "page": 3,
    "span": "total exposure CHF 48'200"
  },
  "confidence": "high"
},
{
  "field": "root_cause",
  "value": null,
  "reason": "not stated in document"
}
```

### Checkable without a model

The quoted sentence either exists in the source document or it does not. That is a text search, not a judgement — and a value whose evidence is missing fails automatically.

### Review becomes fast

The reviewer opens page 3 instead of re-reading the file. This is what makes checking a few hundred documents realistic.

### "Not stated" is a valid answer

A field the model cannot evidence is returned empty with a reason, never guessed. Empty and low-confidence fields are the ones sent for human review.

MEASUREMENT

# How we know the output is correct

Most of what an audit extracts is objectively checkable. The model judge is the exception, not the method.

| WHAT WE EXTRACT | HOW IT IS CHECKED | NEEDS A MODEL? |
|---|---|---|
| Amounts, dates, references | Exact match against the verified answer, after normalising format | No |
| Status, severity, category | Must be one of the allowed values, and the expected one | No |
| Evidence spans | The quoted sentence must exist in the source document | No |
| Narrative cause and summary | Model judge, calibrated against scores given by our own reviewers | Yes |

### Structure first

Every output is validated against a declared schema before anything reads it. A malformed answer fails at the stage that produced it, not three steps later.

### Scored field by field

We never publish one blended accuracy. A field at 62% and a field at 99% need completely different work.

### Judges are measured too

Before a model judge is trusted, we score the same examples by hand and measure agreement. An uncalibrated judge is just a second opinion.

THE REVIEW LOOP

# Where the correct answers come from

Reviewing results is how the reference set gets built. There is no separate labelling project.

### Run

produces results plus a review file

### Review

analyst marks each field correct, wrong, or supplies the right value

### Reference set

corrections are merged into the verified answers

### Every future run

scored automatically against them

*The loop closes: what a reviewer corrects once is checked automatically for ever after.*

### The work becomes the asset

The reading and checking analysts already do is captured instead of discarded. Nobody is asked to label data as a separate task.

### Errors cannot come back

Every mistake corrected once becomes a permanent regression test. The set only ever gets stronger.

### It compounds

After a handful of audits the reference set is large enough to detect drift on its own, without anyone reading every result.

CHANGE CONTROL

# Nothing ships without a measured comparison

Changing a prompt or a model is a decision with a report behind it, not a leap of faith.

| FIELD | CURRENT | CANDIDATE MODEL | CHANGE |
|---|---|---|---|
| `loss_amount` | 99.1% | 99.4% | **+0.3** |
| `resolution_date` | 98.2% | 98.2% | — |
| `root_cause` | 62.4% | 71.8% | **+9.4** |
| `affected_systems` | 88.0% | 79.5% | **-8.5 regression** |

*One model looks better overall and is worse on the field we care about most. Without this table, we would have shipped it.*

### MLflow keeps the record

Every run, every model call, every judge score and every metric is tracked in MLflow — the same platform the company already uses for model work.

### Versions pinned exactly

Model identifiers are fixed in configuration. Nothing silently re-points to a newer version between runs.

### The same gate for everything

Prompt edits, model upgrades and converter changes all pass through this comparison before they are used on real work.

REPRODUCIBILITY

# Built to be re-run and defended

Every output carries its own provenance, and every change re-runs exactly what depends on it.

**Snapshot**

`the issue or PDF as fetched, frozen`

**Extraction**

`depends on snapshot + prompt v1`

**Verified result**

`depends on both of the above`

### A signature per result

Each output is stamped with its inputs, the prompt version, the model and the date it ran.

### Change one prompt, re-run one stage

Everything that does not depend on the change is reused, so iterating stays cheap.

### Any past audit can be replayed

An analysis from six months ago reproduces exactly, from its own pinned snapshots.

IN PRACTICE

# A full run, end to end

Four commands. The plan and the score are the two you will look at every time.

terminal

```
$ dossier sync incident-audit
  incidents-jira          420 known · 18 changed · 2 new   -> 20 fetched
  procedures-confluence    31 known ·  0 changed           ->  0 fetched
  ledger updated · 21 new snapshots · 0 tombstoned

$ dossier run incident-audit --plan
  extract    21 to run · 404 cached      (new or changed snapshots)
  verify     21 to run · 404 cached      (upstream changed)
  report      1 to run                   (inputs changed)

$ dossier run incident-audit
  extract  ####################  21/21    2 fields returned empty
  verify   ####################  21/21    3 corrections applied
  report   runs/2026-08-06/report.md  ·  review.jsonl (7 to check)

$ dossier eval incident-audit --gold gold/incidents
  52 documents · 9 fields · overall 94.1%  ·  root_cause 62.4%  <- weakest
```

SCALABILITY

# What it takes to extend it

The cost of the second, third and tenth use case is what decides whether this is worth building.

A NEW AUDIT

## A folder and a config block

- Write the prompts for the new question
- Point it at sources that already sync
- Agree a reference set with the domain expert
- Written by an analyst, no developer needed

A NEW SOURCE SYSTEM

## One small connector

- Implement three functions: list, fetch, attachments
- Ledger, hashing and caching come for free
- Every existing audit can use it immediately
- A developer task, but a contained one

BUILD CHOICES

# Technical foundation

Chosen for the people who will maintain it, and for what we already run.

## Python

for the engine

The document tooling we need — PDF, OCR, image handling — lives here first.

Every source system and model provider ships a Python client.

It is the language our analysts and data people can already read and edit.

### MLflow for tracking and judges

Runs, traces, judge scores and metrics land in the platform the company already uses for model work — one system, not two.

### YAML for sources and pipelines

What to collect and what to ask — readable without any coding knowledge.

### Declared schemas between stages

Every output validated on the way out, so a bad answer fails where it was produced.

### Git for prompts and configuration

Versioned like code, with review before a change lands.

HOW WE BUILD IT

# Driven by specifications

The specification is the deliverable. The code is generated from it, and regenerated when it changes.

```
specs/

001-source-connectors.md
002-ledger-and-sync.md
003-pipeline-format.md
004-step-resolution.md
005-caching-and-keys.md
006-output-schemas.md
007-scoring-and-gold.md

Each spec ends with a
numbered "must hold" list.
That list is the test file.
```

01

### Someone edits a spec

A rule changes — in plain language, in a reviewed file.

02

### The code is regenerated

The implementation follows the spec, including the tests it names.

03

### The suite proves it

Sync, caching and provenance are ordinary software with exact answers, tested before the code exists.

04

### The reference set proves the rest

Anything model-dependent is measured against verified answers, never assumed.

PLAN

# Proposed rollout

Prove it can be measured before we widen it.

PHASE 1

## Prove it is correct

- Manual folder source only
- Grounded extraction with evidence spans
- A reference set of 50 incidents agreed with the domain expert

PHASE 2

## Connect the sources

- Jira connector with incremental sync
- Confluence export with images preserved
- MLflow tracking and the comparison gate

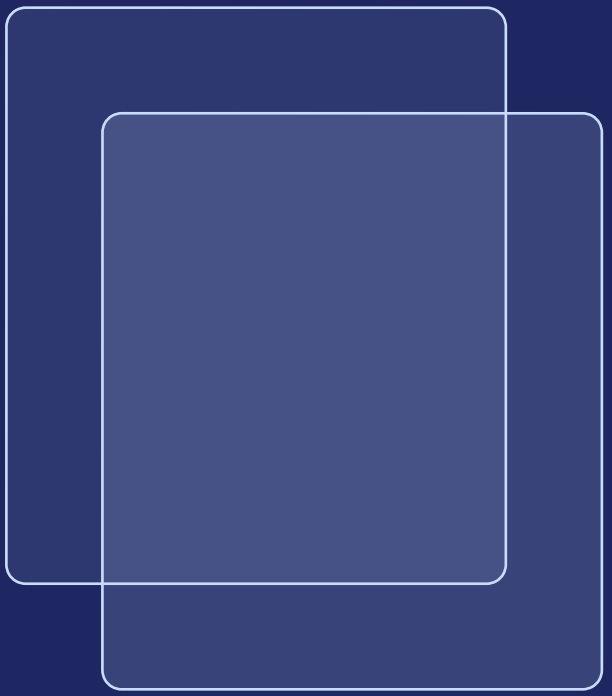
PHASE 3

## Open it up

- Other teams bring their own prompts
- A second, unrelated use case
- Decision on where the models run

WHAT WE NEED

# Four decisions



**01**

## A first official use case

Incident report auditing, with a domain expert who can sign off what the correct answers are.

**02**

## Time to build the core

A bounded effort to deliver the engine, its specs, its test suite and the first reference set.

**03**

## Read access to the source systems

A service account for Jira and Confluence, scoped to the projects and spaces in question.

**04**

## A direction on where models run

Local hosting or a hosted API — this shapes the cost model, not the architecture.

*Ask 01 is the one that matters. Without agreed correct answers, everything else is unverifiable.*